RAG CHATBOT

Multimodal AI Assistant

Complete Project Documentation

Student	TALHA SHAHZAD
Subject	Mobile Application Development
Semester	6th Semester - Ghazi University, Dera Ghazi Khan.
Date	13 - April 2026

1. Project Overview

RAG Chatbot is a Multimodal AI-powered assistant built as a semester project for the Mobile Application Development course. It combines a Flutter-based frontend with a FastAPI Python backend to deliver an intelligent conversational experience capable of answering questions from both general knowledge and uploaded documents.

The system uses Retrieval-Augmented Generation (RAG) — a technique that allows the AI to search through user-uploaded documents and use that content as context when answering questions, making responses grounded in real document data rather than hallucinated information.

1.1 Key Highlights

- Multimodal document support: PDF, DOCX, PPTX, XLSX, CSV, images (OCR), and MP4 video (speech-to-text)
- Firebase Authentication with Email/Password and Google Sign-In
- Real-time chat with AI responses powered by Groq LLM (Llama 3.3 70B)
- Text-to-Speech audio responses using Google TTS (gTTS)
- Two chat modes: General Knowledge and Document-Based Q&A
- Chat history stored in Firestore with session grouping
- Dark theme professional UI inspired by Claude and ChatGPT
- FAISS vector database for fast semantic document search
- HuggingFace sentence embeddings for document indexing

1.2 Technology Stack

Layer	Technology	Purpose
Frontend	Flutter (Dart)	Cross-platform mobile/web UI
Backend	FastAPI (Python)	REST API server
LLM	Groq API — Llama 3.3 70B	AI language model
Vector DB	FAISS (Facebook AI)	Semantic document search
Embeddings	sentence-transformers/all-MiniLM-L6-v2	Text vectorization
Auth	Firebase Authentication	User login and sessions
Database	Cloud Firestore	Chat history storage
TTS	gTTS (Google Text-to-Speech)	Audio responses
State Mgmt	Provider (Flutter)	App state management
HTTP Client	Dio (Flutter)	API communication
Routing	GoRouter (Flutter)	Screen navigation

2. System Architecture

The application follows a clean client-server architecture with three main layers: the Flutter frontend, the FastAPI backend, and the Firebase cloud services.

2.1 Architecture Overview

> **Architecture Pattern**
> The system uses a RESTful API architecture where the Flutter client communicates with the FastAPI server over HTTP. Firebase handles authentication and data persistence independently, running in parallel with the main API.

2.2 Backend Architecture

The FastAPI backend is the core processing engine of the application. It handles all AI operations, document processing, and audio generation.

Module	File	Responsibility
Main Application	app.py	FastAPI app, routes, CORS middleware
RAG Engine	rag_engine.py	LLM integration, prompt management, context handling
Document Utils	document_utils.py	PDF, DOCX, PPTX, Excel, CSV, image text extraction
Vector Store	vectorstore_utils.py	FAISS index creation, loading, searching, resetting
Audio Utils	audio_utils.py	gTTS text-to-speech generation
Video Utils	video_utils.py	MP4 audio extraction and speech recognition

2.3 Frontend Architecture

The Flutter frontend follows the Provider pattern for state management and GoRouter for navigation. All screens are organized in a feature-based folder structure.

Layer	Components	Description
Screens	Chat, History, Upload, Settings, Auth	User interface pages
Providers	ChatProvider, AuthProvider, ThemeProvider, UploadProvider	Business logic and state
Services	ApiService, FirestoreService	External communication
Models	MessageModel	Data structures
Core	AppTheme, AppColors, AppRouter	App-wide configuration

2.4 Data Flow

1. User sends a message in the chat screen
2. ChatProvider calls ApiService.sendMessage() with question, language, and mode
3. ApiService makes POST request to FastAPI /chat endpoint via Dio
4. FastAPI checks the mode — if Document mode, queries FAISS vectorstore for relevant chunks
5. Context chunks are passed to rag_engine.ask_llm() along with conversation history
6. Groq LLM processes the prompt and returns an answer
7. gTTS generates an MP3 audio file from the answer text
8. Backend returns JSON with answer text and audio URL
9. Flutter displays the message bubble and shows audio player widget
10. Message is saved to Firestore for history

3. Backend Documentation

3.1 API Endpoints

Endpoint	Method	Parameters	Description
/chat	POST	question, language, mode	Send message, get AI response + audio URL
/upload	POST	file (multipart)	Upload document for indexing into FAISS
/reset	POST	none	Clear FAISS index and conversation history
/audio/{filename}	GET	filename	Serve generated MP3 audio files
/docs	GET	none	FastAPI Swagger UI documentation
/test-index	GET	none	Debug endpoint to verify FAISS index contents

3.2 Chat Endpoint Detail

The /chat endpoint is the core of the application. It accepts three query parameters and returns an AI-generated response.

Request Parameters

Parameter	Type	Default	Description
question	string	required	The user's question or message
language	string	english	Response language: 'english' or 'urdu'
mode	string	general	Chat mode: 'general' (knowledge) or 'document' (RAG)

Response Format

```
{ "answer": "AI response text here...", "audio_url":
"http://localhost:8000/audio/abc123.mp3" }
```

3.3 Document Processing Pipeline

When a file is uploaded via POST /upload, it goes through the following processing stages:

11. File received as multipart form data by FastAPI
12. File type detected from extension (.pdf, .docx, .pptx, .xlsx, .csv, .png, .jpg, .mp4)
13. Text extraction performed by the appropriate utility function
14. PDF: PyPDF2 text extraction, falls back to OCR via pytesseract if no text found
15. DOCX: python-docx paragraph extraction

16. PPTX: python-pptx slide text extraction
17. Excel/CSV: pandas DataFrame converted to string
18. Images: pytesseract OCR
19. MP4: moviepy audio extraction, then SpeechRecognition Google API
20. Extracted text split into 900-character chunks with 150-character overlap
21. Chunks embedded using HuggingFace sentence-transformers model
22. Embeddings stored in FAISS local index (faiss_index/ directory)

3.4 RAG Engine

The RAG engine in rag_engine.py manages prompt construction and LLM communication. It uses different prompt templates depending on the chat mode.

Mode	Context Used	Behavior
general	None	Answers from Llama 3.3 70B general knowledge
document (with index)	Top 5 FAISS chunks	Answers strictly from document content
document (no index)	None	Tells user to upload a document first

3.5 Environment Configuration

The backend requires a .env file in the project root with the following variables:

```
GROQ_API_KEY=your_groq_api_key_here
HF_TOKEN=your_huggingface_token_here
```

> **Getting API Keys**
> Groq API key: Sign up free at console.groq.com. HuggingFace token: Create account at huggingface.co, go to Settings > Access Tokens.

4. Frontend Documentation

4.1 Project Structure

```
lib/
├── main.dart                    Entry point, Firebase init, providers
├── firebase_options.dart        Auto-generated Firebase config
├── core/
│   ├── theme/
│   │   ├── app_theme.dart       Dark and light theme definitions
│   │   └── colors.dart          AppColors constants
│   └── router/
│       └── app_router.dart      GoRouter with auth guard
├── models/
│   └── message_model.dart       Chat message data model
├── providers/
│   ├── auth_provider.dart       Firebase auth state management
│   ├── chat_provider.dart       Chat messages, mode, language
│   ├── theme_provider.dart      Dark/light theme toggle
│   ├── upload_provider.dart     File upload state
│   └── history_provider.dart    Chat history state
├── services/
│   ├── api_service.dart         Dio HTTP client for FastAPI
│   └── firestore_service.dart   Firestore read/write operations
└── screens/
    ├── splash/splash_screen.dart
    ├── auth/
    │   ├── login_screen.dart
    │   └── register_screen.dart
    ├── home/home_screen.dart    Bottom navigation shell
    ├── chat/
    │   ├── chat_screen.dart
    │   └── widgets/
    │       ├── message_bubble.dart
    │       ├── chat_input_bar.dart
    │       ├── typing_indicator.dart
    │       └── audio_player_widget.dart
    ├── history/history_screen.dart
    ├── upload/upload_screen.dart
    └── settings/settings_screen.dart
```

4.2 Screens Description

Screen	File	Features
Splash	splash_screen.dart	App logo, auto-redirects based on auth state
Login	login_screen.dart	Email/password login, Google Sign-In, error display
Register	register_screen.dart	New account creation with name, email, password

Screen	File	Features
Chat	chat_screen.dart	Message list, typing indicator, mode badge in app bar
History	history_screen.dart	Session grouping, expand to view full conversation, swipe to delete
Upload	upload_screen.dart	File picker, upload progress, result card, FAISS reset
Settings	settings_screen.dart	Dark mode toggle, chat mode, language, sign out, clear history

4.3 State Management

The application uses the Provider package for state management. All providers extend ChangeNotifier and are registered at the app root in main.dart using MultiProvider.

Provider	State Managed	Key Methods
AppAuthProvider	Current user, loading, error	signInWithEmail(), signInWithGoogle(), signOut()
ChatProvider	Messages list, thinking, mode, language	sendMessage(), clearMessages(), setMode(), setLanguage()
ThemeProvider	ThemeMode (dark/light)	toggle(), persisted in SharedPreferences
UploadProvider	Uploading state, result, filename	upload() for mobile, uploadBytes() for web
HistoryProvider	Session list	setSessions(), clear()

4.4 Navigation (GoRouter)

Navigation uses GoRouter with an auth guard redirect. The router checks Firebase auth state and redirects automatically.

Route	Screen	Auth Required
/splash	SplashScreen	No
/auth/login	LoginScreen	No (redirects to /home if logged in)
/auth/register	RegisterScreen	No
/home	HomeScreen (bottom nav)	Yes (redirects to /auth/login if not logged in)

4.5 Key Flutter Packages

Package	Version	Purpose
firebase_core	^3.3.0	Firebase initialization

Package	Version	Purpose
firebase_auth	^5.1.4	Authentication
cloud_firestore	^5.2.0	Chat history database
google_sign_in	^6.2.1	Google OAuth login
provider	^6.1.2	State management
go_router	latest	Navigation and routing
dio	^5.5.0	HTTP client for API calls
file_picker	^8.0.0	Document file selection
just_audio	^0.9.40	Audio playback for TTS responses
flutter_markdown	^0.7.1	Markdown rendering in chat bubbles
shared_preferences	^2.3.1	Theme preference persistence
timeago	^3.6.1	Relative time display in history
cloud_firestore	^5.2.0	Real-time chat history sync

5. Firebase Configuration

5.1 Firebase Services Used

Service	Purpose	Configuration
Firebase Authentication	User login and session management	Email/Password + Google Sign-In enabled
Cloud Firestore	Chat message history storage	Test mode, us-central1 region
Firebase Core	Base SDK initialization	Auto-configured via flutterfire CLI

5.2 Firestore Data Structure

```
users/ (collection)
  {userId}/ (document)
    messages/ (sub-collection)
      {messageId}/ (document)
        id: string
        role: 'user' | 'assistant'
        content: string
        timestamp: ISO 8601 string
        audioPath: string | null
        isError: boolean
```

5.3 FlutterFire CLI Setup

Firebase is connected to Flutter using the FlutterFire CLI. This automatically generates firebase_options.dart and configures all platform-specific files.

23. Install Firebase CLI: npm install -g firebase-tools
24. Login: firebase login
25. Install FlutterFire CLI: dart pub global activate flutterfire_cli
26. Run in project folder: flutterfire configure
27. Select your Firebase project and target platforms
28. CLI auto-generates lib/firebase_options.dart

6. Installation & Setup Guide

6.1 Prerequisites

Tool	Version	Download
Flutter SDK	3.x or later	flutter.dev
Python	3.9 or later	python.org
Node.js	18.x or later (for Firebase CLI)	nodejs.org
Android Studio	Latest	developer.android.com/studio
VS Code	Latest	code.visualstudio.com
Git	Latest	git-scm.com

6.2 Backend Setup

Step 1 — Clone / Navigate to backend folder

```
cd your_backend_folder
```

Step 2 — Install Python dependencies

```
pip install -r requirements.txt
```

Step 3 — Create .env file

```
GROQ_API_KEY=your_key_here
HF_TOKEN=your_token_here
```

Step 4 — Run the server

```
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

Step 5 — Verify it is running

Open browser and visit: http://localhost:8000/docs — you should see the Swagger UI.

6.3 Frontend Setup

Step 1 — Navigate to Flutter project

```
cd rag_chatbot_app
```

Step 2 — Install Flutter dependencies

```
flutter pub get
```

Step 3 — Configure Firebase

```
flutterfire configure
```

Step 4 — Enable Developer Mode (Windows)

```
start ms-settings:developers
```

Step 5 — Run the app

```
flutter run
```

6.4 Requirements.txt (Backend)

```
fastapi
uvicorn
python-multipart
aiofiles
langchain
langchain-groq
langchain-community
langchain-text-splitters
faiss-cpu
sentence-transformers
python-docx
PyPDF2
pdf2image
pytesseract
Pillow
pandas
openpyxl
xlrd
python-pptx
moviepy
SpeechRecognition
gTTS
python-dotenv
```

7. Feature Documentation

7.1 Chat Feature

The chat screen is the primary interface of the application. Users can send text messages and receive AI responses with optional audio playback.

General Mode

In General mode the AI answers from its training knowledge (Llama 3.3 70B). This is suitable for programming questions, general knowledge, math, explanations, and any question not related to uploaded documents.

Document Mode

In Document mode the AI retrieves the top 5 most relevant chunks from the FAISS vector index and uses them as context to answer the question. This is ideal for asking about the content of uploaded files.

Audio Responses

Every AI response is converted to speech using Google TTS (gTTS). The audio file is saved on the backend server and served as a static file. The Flutter frontend includes an AudioPlayerWidget powered by just_audio that allows users to play, pause, and replay the response.

7.2 Document Upload Feature

The Upload screen allows users to select and upload files from their device. The system supports multiple file formats and automatically extracts text content for indexing.

Format	Extension	Extraction Method
PDF	.pdf	PyPDF2 text extraction, OCR fallback for scanned PDFs
Word Document	.docx	python-docx paragraph extraction
PowerPoint	.pptx	python-pptx shape text extraction
Excel	.xlsx, .xls	pandas DataFrame to string conversion
CSV	.csv	Row-by-row text concatenation
Images	.png, .jpg, .jpeg	pytesseract OCR
Video	.mp4	moviepy audio extraction + Google Speech Recognition

Web Upload Note
On Flutter Web, files are uploaded using bytes (not file path) because the web platform does not provide direct filesystem access. The withData: true flag is set in FilePicker to retrieve file bytes.

7.3 Chat History Feature

All messages are stored in Cloud Firestore in real time. The History screen reads from Firestore and groups messages into conversation sessions based on time gaps greater than 60 minutes.

- Sessions displayed as cards with message count and relative time
- Tap any session card to expand and view the full conversation
- Each message shows the role (You / AI), content, and timestamp
- Swipe left on any message to delete it from Firestore
- Delete button on session card removes the entire conversation
- Search bar filters sessions by content

7.4 Settings Feature

Setting	Options	Behavior
Dark Mode	On / Off	Toggles between dark and light theme, persisted in SharedPreferences
Chat Mode	General / Document	Switches AI response mode, shown as badge in chat app bar
Response Language	English / Urdu	Changes TTS audio language and instructs LLM on response language
Clear Chat History	Confirmation dialog	Deletes all messages from Firestore and clears local provider state
Sign Out	Immediate action	Signs out from Firebase Auth and Google, navigates to login

7.5 FAISS Reset Feature

The Upload screen includes a Danger Zone section with a Reset Knowledge Base button. This calls the POST /reset API endpoint which deletes the faiss_index/ directory on the server and clears the in-memory conversation history. Use this when you want to remove all previously uploaded documents and start fresh.

8. Security Considerations

8.1 Authentication

- All screens except Login and Register are protected by the GoRouter auth guard
- Firebase Auth tokens are managed automatically by the Firebase SDK
- Google Sign-In requires SHA-1 fingerprint registered in Firebase Console for Android
- Auth state changes automatically redirect the user via GoRouterRefreshStream

8.2 Firestore Rules

The current Firestore rules are set to test mode which allows all reads and writes for 30 days. Before production deployment update the rules to restrict access to authenticated users only:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/{document=**} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId;
    }
  }
}
```

8.3 API Security

- CORS middleware is configured to allow all origins during development
- For production, restrict allow_origins to your specific domain
- API keys are stored in .env file and loaded via python-dotenv
- The .env file must never be committed to version control

9. Troubleshooting Guide

Issue	Cause	Solution
CORS error in browser	Backend missing CORS middleware	Add CORSMiddleware to main.py before routes
Audio 404 Not Found	Wrong audio URL or static files not mounted	Mount StaticFiles for audio_files/ directory in main.py
File upload fails on web	Using file path instead of bytes	Use file.bytes with withData: true in FilePicker
Document mode says not available	FAISS index empty or not loaded	Upload a document first, verify at /test-index endpoint
Google Sign-In fails on Android	SHA-1 not registered in Firebase	Run gradlew signingReport, add SHA-1 to Firebase Console
flutter run fails (Windows)	Developer Mode disabled	Run: start ms-settings:developers
FlutterFire configure not found	FlutterFire CLI not in PATH	Add Pub cache bin to Windows PATH environment variable
API connection refused on phone	Using localhost on mobile device	Use PC local IP address (192.168.x.x) for mobile
No readable content in upload	File is empty or unsupported encoding	Try different file, check backend logs for extraction errors
History screen empty	Firestore messages collection empty	Send a chat message first, verify Firestore in Firebase Console

10. Future Improvements

10.1 Planned Features

- Multi-session chat management with named conversation threads
- Support for multiple simultaneous document indexes
- Real-time streaming of AI responses (Server-Sent Events)
- Voice input using device microphone with Whisper transcription
- Android and iOS native app builds with proper permissions
- User profile editing and avatar upload
- Export chat history as PDF or Word document
- Shareable chat links

10.2 Performance Improvements

- Response caching for repeated questions
- Background document processing queue for large files
- Lazy loading of chat history with pagination
- Image compression before OCR processing

10.3 Production Readiness

- Deploy FastAPI on cloud server (Railway, Render, or AWS EC2)
- Add HTTPS with SSL certificate via Nginx + Let's Encrypt
- Update CORS to restrict to production domain only
- Update Firestore security rules for production
- Add rate limiting to API endpoints
- Set up monitoring and logging with Sentry or similar

11. Conclusion

The RAG Chatbot project successfully demonstrates the integration of modern AI technologies with a production-quality mobile application. The system combines Retrieval-Augmented Generation, large language models, vector databases, Firebase cloud services, and a Flutter cross-platform frontend into a cohesive and functional application.

The project covers advanced concepts in mobile development including state management with Provider, real-time database integration with Firestore, authentication flows with Firebase Auth, and REST API communication with Dio. On the backend side it demonstrates practical applications of NLP, document processing, embeddings, and semantic search using FAISS.

The dark-themed, Claude-inspired UI provides a professional and intuitive user experience with features like animated typing indicators, collapsible chat history sessions, audio playback, and real-time document mode switching.

> **Project Summary**
> A full-stack multimodal RAG chatbot with Flutter frontend + FastAPI backend + Firebase auth + Firestore history + FAISS vector search + Groq LLM + gTTS audio responses. Supports PDF, DOCX, PPTX, Excel, CSV, image OCR, and MP4 video document types.